SENATI

VERSIÓN MEJORADA DE EJERCICIOS ANGULAR 19 + PWA

Dos retos de ampliación basados en los proyectos del documento original

PROYECTO 1 · Backus Ventas PWA
PROYECTO 2 · SENATI TaskHub PWA

Angular 19 · TypeScript · Angular Service Worker · Cache Storage · IndexedDB

Instructor: Jorge Luque Chambi

Presentación y alcance

Este material propone una mejora incremental para cada uno de los dos proyectos identificados en el PDF entregado: Backus Ventas PWA (nivel básico) y SENATI TaskHub PWA (nivel intermedio). El estudiante debe conservar el proyecto base y añadir funcionalidades nuevas que puedan probarse.

Criterio técnico transversal
El proyecto base no incorpora una API empresarial real. No se debe afirmar que un pedido o una entrega se envió a un servidor si únicamente se mostró un mensaje. Diferenciar entre guardado local, pendiente de sincronización y sincronizado con servidor.

Cómo trabajar

- Analizar el enunciado y planificar antes de programar.
- Trabajar por incrementos pequeños y probar cada cambio.
- Separar componentes y servicios; preferir interfaces TypeScript a any.
- Probar online y offline con DevTools.
- Entregar código fuente, evidencias y README con instrucciones.

Resumen de retos

Proyecto base	Mejora propuesta	Nivel / tiempo
Backus Ventas PWA	Catálogo con búsqueda y filtros, carrito, cálculo de importes, historial local de pedidos y estados de conectividad.	Básico–intermedio / 180 min
SENATI TaskHub PWA	Bandeja de entregas offline, formulario validado, consulta de pendientes, contador y control honesto de sincronización.	Intermedio / 180 min

PROYECTO 1 · Backus Ventas PWA

Reto: Catálogo inteligente y pedidos confiables sin conexión

Elemento	Especificación
Proyecto de partida	Angular 19, catálogo JSON, estado online/offline e IndexedDB.
Problema	El vendedor necesita encontrar productos rápidamente, registrar varias unidades y revisar pedidos guardados cuando pierde conexión.
Objetivo	Mejorar el catálogo y el registro de pedidos con validación, persistencia y estados transparentes.
Producto esperado	PWA con búsqueda, filtros, carrito simple, cálculo de total, historial de pedidos y estados.
Tiempo sugerido	180 minutos.

A. Funcionalidades obligatorias

- Buscar productos por nombre sin distinguir mayúsculas/minúsculas.
- Filtrar por categoría y poder limpiar filtros.
- Seleccionar cantidad por producto; aceptar solo enteros mayores que cero.
- Mostrar resumen del pedido: producto, precio unitario, cantidad y subtotal por línea.
- Calcular el total sumando todos los subtotales.
- Impedir registrar pedidos vacíos o con cantidades inválidas.
- Guardar en IndexedDB cada pedido con fecha, líneas, total, origen y estado inicial PENDIENTE.
- Mostrar historial local y permitir consultar el detalle de cada pedido.
- Mostrar conectividad y un mensaje coherente: si no hay API, indicar que el pedido quedó guardado en el dispositivo, no enviado al servidor.
- Capturar errores de persistencia; no mostrar éxito si la operación falla.

B. Reglas y aceptación

ID	Regla	Resultado verificable
BV-01	Búsqueda por nombre.	Al escribir “pilsen”, se muestran coincidencias.
BV-02	Búsqueda y categoría combinables.	Solo se muestran productos que cumplen ambos filtros.
BV-03	Cantidad entera positiva.	Se rechazan cero, negativos, decimales y valores no numéricos.
BV-04	Subtotal = precio × cantidad.	Los importes aparecen con dos decimales.
BV-05	Persistencia antes de confirmar.	Tras recargar, el pedido sigue en el historial.
BV-06	Sin API no hay sincronización real.	El pedido conserva estado PENDIENTE.
BV-07	Error de IndexedDB.	Se informa el error y no se confirma éxito.

C. Modelo sugerido

```
export interface LineaPedido {
  productoId: number;
  producto: string;
  precioUnitario: number;
  cantidad: number;
  subtotal: number;
}
export type EstadoPedido = 'PENDIENTE' | 'SINCRONIZADO' | 'ERROR';
export interface Pedido {
  id?: number;
  fecha: string;
  lineas: LineaPedido[];
  total: number;
  estado: EstadoPedido;
  origen: 'ONLINE' | 'OFFLINE';
}
```

Decisión de diseño
Para comprobar la persistencia de forma consistente, guardar primero el pedido en IndexedDB. La sincronización con un servidor real es una ampliación opcional y requiere API.

D. Secuencia de desarrollo

1. Revisar CatalogoService y StorageService; reutilizar y refactorizar lo necesario.
2. Definir interfaces Producto, LineaPedido y Pedido.
3. Implementar búsqueda y filtro por categoría.
4. Construir el resumen, validar cantidades y calcular importes.
5. Ampliar StorageService con métodos para guardar y listar pedidos.
6. Crear historial y detalle del pedido.
7. Añadir mensajes de validación, confirmación y error.
8. Compilar para producción y probar online/offline.

E. Plan de pruebas

Prueba	Procedimiento	Aceptación
P1	Buscar por parte del nombre.	La lista se filtra correctamente.
P2	Combinar búsqueda y categoría.	Solo aparecen coincidencias válidas.
P3	Crear pedido con dos productos y cantidades distintas.	Subtotales y total correctos.
P4	Guardar pedido vacío o cantidad cero.	Se impide el registro.
P5	Guardar, recargar y abrir historial.	El pedido persiste completo.
P6	Activar Offline y guardar pedido.	Queda en IndexedDB como PENDIENTE.

P7	Inspeccionar Application → IndexedDB.	Se observan base de datos y registros.

F. Evidencias

- Captura del catálogo con búsqueda y filtro.
- Captura del resumen con cantidades, subtotales y total.
- Captura del historial de pedidos.
- Captura de IndexedDB con un pedido persistido.
- Captura en modo Offline con estado PENDIENTE.
- README.md con pasos, mejoras y limitaciones.

G. Rúbrica (100 puntos)

Criterio	Indicador	Puntos
Búsqueda y filtros	Funcionan individualmente y combinados.	15
Carrito y cálculos	Validaciones, subtotales y total correctos.	20
IndexedDB	Guarda y recupera pedidos estructurados.	25
Estados online/offline	Mensajes claros, sin falsas afirmaciones de sincronización.	15
Pruebas y evidencias	Capturas claras y reproducibles.	15
Calidad técnica	Tipos, organización y diseño usable.	10
TOTAL		100

H. Reflexión

9. ¿Por qué persistir un pedido antes de confirmar el guardado?
10. ¿Qué diferencia hay entre Cache Storage e IndexedDB?
11. ¿Qué debería enviarse a una API cuando se implemente la sincronización?
12. ¿Cómo evitarías duplicar pedidos tras varios intentos?

PROYECTO 2 · SENATI TaskHub PWA

Reto: Bandeja de entregas offline con validación y control de estado

Elemento	Especificación
Proyecto de partida	Angular 19, tareas desde JSON, detección online/offline e IndexedDB para entregas pendientes.
Problema	El estudiante necesita saber qué tarea seleccionó, consultar entregas guardadas y distinguir pendientes de entregas realmente sincronizadas.
Objetivo	Convertir el almacenamiento inicial en un flujo de entregas local, consultable, validado y con estados explícitos.
Producto esperado	Lista filtrable, formulario de entrega, bandeja local, validaciones, eliminación confirmada y reintento controlado.
Tiempo sugerido	180 minutos.

A. Funcionalidades obligatorias

- Mostrar curso, título, fecha de entrega y estado académico de cada tarea.
- Buscar por título y filtrar por curso; permitir combinar ambos filtros.
- Seleccionar una tarea y redactar comentario de entrega.
- Validar tarea seleccionada y comentario no vacío; mostrar mensajes específicos.
- Guardar tareaId, comentario, fechaRegistro, estadoSincronizacion e identificador local en IndexedDB.
- Crear una bandeja “Mis entregas locales” que recupere datos después de recargar.
- Permitir ver detalle y eliminar una entrega solo tras confirmación.
- Mostrar contador de entregas pendientes y estado de conexión.
- Al recuperar la conexión, actualizar el indicador y ofrecer reintento; sin API real, mantener PENDIENTE y explicarlo.

B. Reglas y aceptación

ID	Regla	Resultado verificable
TH-01	Búsqueda y filtro combinables.	Lista solo tareas coincidentes.
TH-02	Tarea y comentario obligatorios.	No se guarda formulario inválido.
TH-03	La entrega guarda tarea relacionada y fecha.	La bandeja muestra datos tras recargar.
TH-04	Nueva entrega empieza PENDIENTE.	IndexedDB muestra el estado inicial.
TH-05	Contador derivado de registros persistidos.	El número coincide con pendientes.
TH-06	Online no significa sincronizado.	No cambia a SINCRONIZADO sin confirmación de API.

TH-07	Eliminación confirmada.	Cancelar conserva; confirmar elimina.
TH-08	Errores de IndexedDB informados.	No se pierde silenciosamente el formulario.

C. Modelo de datos sugerido

```
export type EstadoSincronizacion = 'PENDIENTE' | 'SINCRONIZADO' | 'ERROR';
export interface EntregaLocal {
  id?: number;
  tareaId: number;
  curso: string;
  tituloTarea: string;
  comentario: string;
  fechaRegistro: string;
  estadoSincronizacion: EstadoSincronizacion;
  intentosSincronizacion: number;
  ultimoError?: string;
}
```

D. Métodos sugeridos para StorageService

Método	Responsabilidad
guardarEntregaLocal(entrega)	Guardar entrega con fecha y estado inicial.
listarEntregas()	Devolver las entregas para la bandeja.
obtenerEntrega(id)	Consultar detalle de una entrega.
eliminarEntrega(id)	Eliminar registro tras confirmación.
contarPendientes()	Contar registros con estado PENDIENTE.
actualizarEstado(id, estado)	Actualizar estado y metadatos tras un intento real.

Importante sobre sincronización
El código inicial puede mostrar un mensaje de envío sin endpoint real. Usa “Con conexión: listo para intentar sincronizar” hasta integrar una API. Solo marca SINCRONIZADO después de una respuesta remota confirmada.

E. Secuencia de desarrollo

1. Revisar TareaService, StorageService y el esquema actual de IndexedDB.
2. Definir EntregaLocal y estados permitidos.
3. Implementar búsqueda por título y filtro por curso.
4. Completar validaciones del formulario.
5. Ampliar StorageService con listar, obtener, eliminar y contar pendientes.
6. Crear la bandeja local y actualizarla después de guardar o eliminar.
7. Añadir confirmación de eliminación y manejo de errores.
8. Probar persistencia al recargar, en Offline y al volver a Online.
9. Documentar qué funciona localmente y qué necesita una API futura.

F. Plan de pruebas

Prueba	Procedimiento	Aceptación
T1	Buscar por título y filtrar curso.	Filtros individuales y combinados correctos.
T2	Enviar formulario vacío.	No se guarda y aparece validación.
T3	Guardar comentario en Offline.	IndexedDB conserva entrega PENDIENTE.
T4	Recargar la PWA.	La bandeja recupera la entrega.
T5	Crear varias entregas.	Contador refleja pendientes reales.
T6	Cancelar y confirmar eliminación.	Cancelar conserva; confirmar elimina.
T7	Cambiar de Offline a Online.	Cambia indicador; no simula sincronización.
T8	Inspeccionar IndexedDB.	Se observan tarea, comentario, fecha y estado.

G. Evidencias

- Captura de tareas con búsqueda y filtro.
- Captura de validación del formulario.
- Captura de bandeja local con al menos dos entregas.
- Captura de IndexedDB con estado y fecha.
- Capturas en Offline y después de recuperar conexión.
- README.md con funcionalidades, pruebas y limitaciones.

H. Rúbrica (100 puntos)

Criterio	Indicador	Puntos
Lista y filtros	Búsqueda y filtro correctos.	15
Formulario y validación	Tarea y comentario obligatorios.	15
IndexedDB	Guarda, lista, consulta y elimina correctamente.	25
Estados y sincronización	Contador correcto y estados honestos.	20
Pruebas y evidencias	Pruebas reproducibles.	15
Calidad técnica	Tipado, responsabilidades separadas, diseño adaptable.	10
TOTAL		100

I. Reflexión

1. ¿Por qué una entrega debe persistir tras recargar el navegador?
2. ¿Qué campos relacionan la entrega con su tarea?
3. ¿Qué riesgo hay al reintentar sincronizar varias veces?
4. ¿Qué validaciones debe repetir el servidor al integrar una API?

Orientaciones técnicas comunes

1. Pruebas correctas de una PWA Angular

- Verificar el Service Worker con una compilación de producción; no asumir que estará activo en ng serve.
- Ejecutar ng build --configuration production y servir la salida con localhost o HTTPS.
- Usar DevTools → Application para revisar Manifest, Service Workers, Cache Storage e IndexedDB.
- Usar DevTools → Network → Offline para simular pérdida de conexión y después recuperar la red.
- Probar la primera visita online y una visita posterior offline; los recursos que nunca se solicitaron o almacenaron no estarán disponibles automáticamente.

2. No confundir los mecanismos

Mecanismo	Uso en estos proyectos	No significa
App Manifest	Nombre, iconos, colores y experiencia de instalación.	Guardar pedidos o entregas.
Service Worker / Cache Storage	Servir recursos y respuestas GET cacheables según configuración.	Cola automática de solicitudes POST.
IndexedDB	Guardar pedidos y entregas estructuradas localmente.	Sincronización remota automática.
navigator.onLine	Indicador aproximado de conectividad del navegador.	Garantía de que servidor/API responde.

3. Requisitos mínimos de calidad

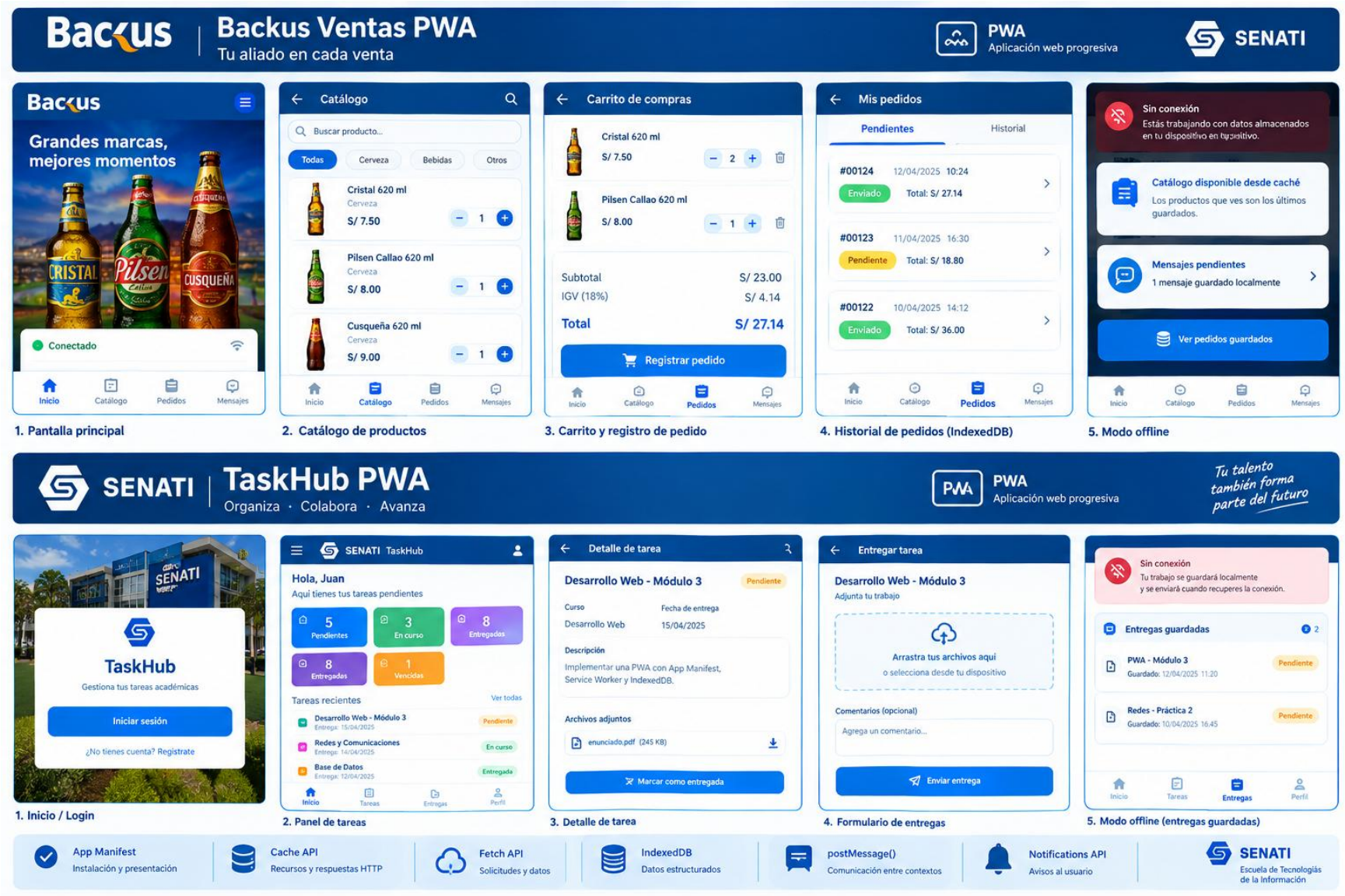
- No mostrar mensajes de éxito engañosos.
- Manejar errores de lectura y escritura de IndexedDB.
- Validar formularios y evitar duplicados accidentales.
- Usar interfaces TypeScript y estados explícitos.
- Mantener interfaz legible en escritorio y móvil.
- No utilizar credenciales ni datos personales reales en los ejemplos.

4. Lista de entrega

- Proyecto Angular fuente o repositorio indicado por el instructor.
- README.md con comandos de ejecución y pasos de prueba.
- Evidencias específicas del proyecto.
- Explicación individual de la mejora y sus limitaciones.

Criterio de aprobación sugerido

La mejora se considera terminada cuando las funcionalidades nuevas trabajan, los datos persisten tras recargar, las pruebas offline están documentadas y la interfaz diferencia guardar localmente de sincronizar con un servidor.


Backus
Backus Ventas PWA
Tu aliado en cada venta
PWA
Aplicación web progresiva
SENATI
Grandes marcas, mejores momentos
Conectado
Inicio
Catálogo
Pedidos
Mensajes
1. Pantalla principal
Catálogo
Buscar producto...
Todas
Cerveza
Bebidas
Otros
Cristal 620 ml
Cerveza
S/ 7.50
Pilsen Callao 620 ml
Cerveza
S/ 8.00
Cusqueña 620 ml
Cerveza
S/ 9.00
2. Catálogo de productos
Carrito de compras
Cristal 620 ml
S/ 7.50
Pilsen Callao 620 ml
S/ 8.00
Subtotal
S/ 23.00
IGV (18%)
S/ 4.14
Total
S/ 27.14
Registrar pedido
3. Carrito y registro de pedido
Mis pedidos
Pendientes
Historial
#00124
12/04/2025 10:24
Enviado
Total: S/ 27.14
#00123
11/04/2025 16:30
Pendiente
Total: S/ 18.80
#00122
10/04/2025 14:12
Enviado
Total: S/ 36.00
4. Historial de pedidos (IndexedDB)
Sin conexión
Estás trabajando con datos almacenados en tu dispositivo en especial.
Catálogo disponible desde caché
Los productos que ves son los últimos guardados.
Mensajes pendientes
1 mensaje guardado localmente
Ver pedidos guardados
5. Modo offline
SENATI
TaskHub PWA
Organiza · Colabora · Avanza
PWA
Aplicación web progresiva
Tu talento también forma parte del futuro
TaskHub
Gestiona tus tareas académicas
Iniciar sesión
¿No tienes cuenta? Regístrate
1. Inicio / Login
SENATI TaskHub
Hola, Juan
Aquí tienes tus tareas pendientes
5 Pendientes
3 En curso
8 Entregadas
8 Entregadas
1 Vencidas
Tareas recientes
Ver todas
Desarrollo Web - Módulo 3
Pendiente
Redes y Comunicaciones
En curso
Base de Datos
Entregada
2. Panel de tareas
Detalle de tarea
Desarrollo Web - Módulo 3
Pendiente
Curso
Desarrollo Web
Fecha de entrega
15/04/2025
Descripción
Implementar una PWA con App Manifest, Service Worker y IndexedDB.
Archivos adjuntos
enunciado.pdf (245 KB)
Marcar como entregada
3. Detalle de tarea
Entregar tarea
Desarrollo Web - Módulo 3
Adjunta tu trabajo
Arrastra tus archivos aquí
o selecciona desde tu dispositivo
Comentarios (opcional)
Agrega un comentario...
Enviar entrega
4. Formulario de entregas
Sin conexión
Tu trabajo se guardará localmente y se enviará cuando recuperes la conexión.
Entregas guardadas
PWA - Módulo 3
Guardado: 12/04/2025 11:20
Pendiente
Redes - Práctica 2
Guardado: 10/04/2025 16:45
Pendiente
5. Modo offline (entregas guardadas)
App Manifest
Instalación y presentación
Cache API
Recursos y respuestas HTTP
Fetch API
Solicitudes y datos
IndexedDB
Datos estructurados
postMessage()
Comunicación entre contextos
Notifications API
Avisos al usuario
SENATI
Escuela de Tecnologías de la Información